

UNIVERSIDADE REGIONAL DE BLUMENAU
CENTRO DE CIÊNCIAS EXATAS E NATURAIS
DEPARTAMENTO DE SISTEMAS E COMPUTAÇÃO
DISCIPLINA: SISTEMAS DISTRIBUÍDOS

FELIPE GROCHOVSKI
ELCIO CLEITON WIPPEL

**MICROCHEFS: ARQUITETURA DE MICROSERVIÇOS
PARA UM SISTEMA DE RESTAURANTE
COM MENSAGERIA, AUTENTICAÇÃO E
RASTREAMENTO EM TEMPO REAL**

Trabalho Prático – Microserviços e Sistemas Distribuídos

BLUMENAU/SC

2025

SUMÁRIO

1. Introdução	3
2. Fundamentação Teórica	3
3. Metodologia	4
4. Desenvolvimento	5
4.1. Visão Geral da Arquitetura	5
4.2. APIs Externas Utilizadas	7
4.3. Rastreamento Geográfico e Simulação de Entrega	8
4.4. Observabilidade, Escalabilidade e Cache	8
4.5. Configuração de APIs e Infraestrutura	9
5. Resultados e Discussão	10
6. Conclusões	10
6.1. O que não foi implementado	11
6.2. Dificuldades Encontradas	11
6.3. Trabalhos Futuros	11
Referências	12
1. Introdução.....	3
2. Fundamentação Teórica.....	3
3. Metodologia.....	4
4. Desenvolvimento	5
4.1. Visão Geral da Arquitetura.....	5
4.2. APIs Externas Utilizadas	7
4.3. Rastreamento Geográfico e Simulação de Entrega	8
4.4. Observabilidade, Escalabilidade e Cache	8
4.5. Configuração de APIs e Infraestrutura	9
5. Resultados e Discussão.....	9
6. Conclusões.....	10
6.1. O que não foi implementado	11
6.2. Dificuldades Encontradas	11
6.3. Trabalhos Futuros	11
Referências	12

1. Introdução

O MicroChefs foi concebido como um sistema de restaurante baseado em microserviços, com o objetivo de demonstrar, em um contexto acadêmico, a separação de responsabilidades entre domínios, a comunicação assíncrona orientada a eventos e o controle de acesso por perfis de usuário. A solução procura reproduzir, em escala simplificada, a experiência de um aplicativo de pedidos, incluindo autenticação, navegação por cardápio, montagem de carrinho, checkout, pagamento via Pix, acompanhamento do status do pedido e apoio às atividades da cozinha e da administração.

A arquitetura adotada distribui as responsabilidades entre serviços independentes. O serviço de usuários centraliza autenticação, cadastro, perfis, endereços, configurações do restaurante e atualização de localização. O serviço de produtos mantém o catálogo e o estoque. O serviço de pedidos atua como orquestrador do fluxo comercial, validando estoque, registrando o pedido, gerando pagamento Pix e publicando eventos. O serviço de cozinha consome os eventos do pedido pago e evolui o status do preparo. Por fim, o serviço de suporte a DLQ registra falhas de processamento e permite correção manual e reenvio das mensagens.

Do ponto de vista de interface, o frontend em Angular concentra as jornadas de cliente, motoboy, cozinha e administrador em uma aplicação única, com roteamento protegido por papéis e comunicação com os microserviços por meio de serviços HTTP, guardas e interceptadores. A proposta geral do projeto é evidenciar boas práticas de modularização, escalabilidade e resiliência, com ênfase em mensageria, persistência segregada e atualização de estado em tempo quase real.

2. Fundamentação Teórica

Microserviços consistem em um estilo arquitetural no qual a aplicação é decomposta em unidades pequenas e independentes, cada uma responsável por um contexto de negócio específico. Essa abordagem favorece isolamento de falhas, escalabilidade seletiva e evolução independente de partes do sistema. Em contrapartida, exige maior cuidado com integração, consistência e observabilidade.

No MicroChefs, a comunicação entre serviços combina chamadas síncronas e assíncronas. As chamadas síncronas são utilizadas quando há dependência imediata de resposta, como na validação de estoque no cadastro de pedidos. Já a mensageria com RabbitMQ é empregada para desacoplar etapas do fluxo, especialmente na transição do pedido para a cozinha e na propagação de atualizações de status. Essa estratégia reduz o acoplamento temporal entre componentes e facilita o tratamento de picos de carga.

A persistência segregada por serviço é outro princípio relevante. Cada microserviço mantém seu próprio banco de dados, o que evita dependência direta de esquema entre domínios distintos. O serviço de usuários utiliza MySQL, enquanto os serviços de pedidos, produtos e DLQ usam PostgreSQL. Essa separação reforça a independência dos módulos e impede que alterações internas em um serviço impactem diretamente os demais.

O projeto também utiliza autenticação baseada em JWT (JSON Web Token). Após o login, o token contém informações de identidade e papel do usuário, permitindo que o frontend e

os serviços protejam rotas e endpoints conforme o perfil. Essa abordagem reduz a necessidade de manutenção de sessão centralizada e simplifica a validação da autorização em múltiplos serviços.

No frontend, a atualização reativa de dados é apoiada por sinais do Angular e por um serviço de tempo real que acompanha mudanças de status, oferecendo feedback rápido sobre o ciclo do pedido. Para observabilidade, o sistema integra Prometheus e Grafana, coletando métricas de latência, vazão e estado das filas. O cache distribuído com Redis reduz a carga sobre o banco de dados relacional em operações de leitura intensiva do catálogo.

3. Metodologia

A construção do MicroChefs seguiu uma abordagem de desenvolvimento incremental, orientada por módulos funcionais. Cada domínio de negócio foi implementado como um serviço independente, com contrato explícito de entrada e saída, permitindo validar a integração em etapas. O trabalho foi organizado a partir dos requisitos do sistema de restaurante, do fluxo de pedido e dos perfis de acesso definidos para cliente, motoboy, cozinha e administrador.

As tecnologias principais adotadas foram:

- Backend de usuários em C# com ASP.NET Core e Entity Framework Core.
- Backend de pedidos, produtos, cozinha e DLQ Support em Java com Spring Boot.
- Frontend em Angular 20 com componentes standalone, Signals, Angular Material, SCSS e módulos de rota.
- RabbitMQ como barramento de eventos assíncronos (hospedado no CloudAMQP em produção).
- PostgreSQL e MySQL como bancos relacionais dedicados por serviço.
- Redis como cache distribuído para o catálogo de produtos.
- Docker e Docker Compose para orquestração local dos serviços.
- Prometheus e Grafana para monitoramento e observabilidade operacional.
- OpenRouteService e Nominatim (OpenStreetMap) para geocodificação e cálculo de rotas reais.
- Leaflet para renderização de mapas interativos no frontend.

A topologia de execução local foi definida no arquivo `docker-compose.yaml`, que declara os serviços, suas portas, variáveis de ambiente, dependências e redes internas. Essa organização permite reproduzir o ambiente completo em desenvolvimento e demonstra a separação operacional entre os módulos.

O fluxo de dados foi modelado em torno do ciclo de vida do pedido: (1) o usuário autenticado cria o pedido no frontend; (2) o serviço de pedidos valida cliente, itens e estoque; (3) o pedido é persistido com status inicial de aguardo de pagamento; (4) em pagamentos imediatos, o estoque é debitado e o pedido avança; (5) em Pix, o pedido recebe chave de

cópia e cola e aguarda confirmação; (6) após confirmação, o serviço de pedidos publica o evento para a cozinha via RabbitMQ; (7) a cozinha processa a fila e atualiza o status; (8) se uma mensagem falhar, ela é enviada à DLQ para correção manual.

4. Desenvolvimento

4.1. Visão Geral da Arquitetura

A arquitetura do sistema foi estruturada com foco em desacoplamento entre domínios. Cada microserviço possui uma responsabilidade delimitada, um banco próprio e um conjunto de endpoints ou consumidores de fila. O diagrama abaixo sintetiza as dependências e os canais de comunicação entre os componentes:

Figura 1 – Diagrama lógico da arquitetura do sistema MicroChefs

Frontend Angular → Usuario Service (HTTP) | Produto Service (HTTP) | Pedido Service (HTTP) | DLQ Support (HTTP)

Pedido Service → Produto Service (HTTP síncrono para validação de estoque)

Pedido Service → RabbitMQ pedido-exchange [pedido-key.pago] → Cozinha Service

Cozinha Service → RabbitMQ pedido-exchange [pedido-key.update] → Pedido Service

Pedido Service → RabbitMQ pedido-exchange [pedido-key.status] → Usuario Service

Qualquer consumer → dead-letter-exchange [dead-message] → dead-letter-queue → DLQ Support

No serviço de usuários (C# / ASP.NET Core), o Program.cs configura CORS, JWT Bearer, Entity Framework Core com MySQL e um consumer em background para eventos de status do pedido. O AuthService realiza login a partir de e-mail e senha, valida a senha por hash, gera tokens assinados e inclui as claims de papel e cliente no JWT. O controlador de autenticação expõe endpoints de login e refresh; o fluxo de refresh reutiliza a identidade do token atual para emitir nova credencial sem exigir novo login.

No serviço de produtos (Java / Spring Boot), o ProdutoController disponibiliza criação, edição, listagem, consulta, exclusão e débito de estoque. A classe ProdutoService adota cache com @Cacheable e @CacheEvict, mantendo o catálogo em memória distribuída por Redis para acelerar leituras frequentes. O método de redução de estoque executa validação de disponibilidade, atualiza o banco e invalida as entradas de cache, garantindo coerência entre persistência e camada de leitura. O trecho a seguir ilustra a invalidation strategy adotada:

```
@Caching(evict = {  
    @CacheEvict(value = "produto", key = "#id"),  
    @CacheEvict(value = "produtos", allEntries = true)  
})
```

```
public RecoveryProdutoDTO editarProduto(long id, CreateProdutoDTO
dto) { ... }
```

Código 1 – Estratégia de invalidação de cache no serviço de produtos

Toda alteração em produto invalida a entrada individual e a listagem geral, evitando que o frontend exiba preço, imagem ou estoque defasados após uma edição administrativa.

No serviço de pedidos, o PedidoService é o núcleo de orquestração. O método criarPedido verifica permissão do cliente, valida a presença de itens, consulta o serviço de produtos para confirmar estoque e calcula o valor total. Caso o pagamento seja em dinheiro ou cartão, o sistema debita o estoque e marca o pedido como pago automaticamente. Caso o pagamento seja Pix, o sistema gera uma chave cópia e cola e mantém o pedido em estado pendente:

```
if
(createPedidoDto.formaDePagamento().equals(FormaDePagamento.PIX)) {
    String txId = String.format("MC%08d", pedido.getId());
    String pixCode = PixUtils.gerarPixCopiaECola(pedido.getId(),
pedido.getValorTotal());
    pedido.setPixCopiaECola(pixCode);
    pedido.setPixTransactionId(txId);
    pedido.setPaymentStatus("PENDENTE");
}
```

Código 2 – Geração do código Pix no serviço de pedidos

Outra decisão central está no controle de transição de estados. O enum StatusPedido define os estados permitidos e os próximos estados válidos a partir de cada etapa, impedindo saltos indevidos e preservando a consistência do fluxo:

```
public enum StatusPedido {
    AGUARDANDO_PAGAMENTO { public List<StatusPedido>
proximosEstados()
        { return List.of(CANCELADO, PAGO); } },
    PAGO { public List<StatusPedido> proximosEstados()
        { return List.of(EM_PREPARO); } },
    EM_PREPARO { public List<StatusPedido> proximosEstados()
        { return List.of(PRONTO); } },
    PRONTO { public List<StatusPedido> proximosEstados()
        { return List.of(AGUARDANDO_ENTREGADOR); } },
    EM_ENTREGA { public List<StatusPedido> proximosEstados()
        { return List.of(ENTREGUE); } }
}
```

Código 3 – Enum de estados do pedido com transições válidas

Depois da confirmação do pagamento Pix, o serviço publica eventos para a cozinha por meio do PedidoProducer, que envia mensagens para o tópico pedido-exchange com chaves de roteamento específicas. O serviço de cozinha consome a fila cozinha-queue, processa o pedido e publica atualizações intermediárias e finais com retry, tratamento de exceções e registro de métrica via Micrometer. Caso ocorra falha, a mensagem é encaminhada para o suporte de DLQ.

No suporte de DLQ, o `SupportConsumer` recebe mensagens da dead-letter-queue, salva o conteúdo original e cria um registro de correção. O `SupportService` permite consultar detalhes da falha, abrir o editor JSON e reenviar a mensagem corrigida para a fila de origem, sem perda definitiva:

```
public void atualizarMensagem(FixedMessageDTO fixedMensagemDto) {
    Mensagem mensagemAtualizada = supportRepository
        .findById(fixedMensagemDto.id())
        .orElseThrow(() -> new MensagemNotFoundException("Mensagem
        não encontrada"));

    mensagemAtualizada.setCorrecaoDocumentada(fixedMensagemDto.correcao
        Documentada());

    mensagemAtualizada.setMensagemCorrigida(fixedMensagemDto.mensagemCo
        rrigida());
    supportRepository.save(mensagemAtualizada);
    mensagemProducer.reenviarJson(fixedMensagemDto);
}
```

Código 4 – Reenvio de mensagem corrigida no DLQ Support

No frontend, o roteamento foi organizado por perfis. Guardas de autenticação e de role protegem as rotas sensíveis, enquanto o `AuthService` mantém a sessão local e sincroniza o `clienteId`:

```
{
  path: "admin",
  canActivate: [authGuard, roleGuard("ADMIN")],
  children: [
    { path: "dashboard", loadComponent: () => DashboardComponent },
    { path: "produtos", loadComponent: () => ProdutosComponent },
    { path: "usuarios", loadComponent: () => UsuariosComponent },
  ]
}
```

Código 5 – Rota protegida por perfil no Angular

4.2. APIs Externas Utilizadas

O sistema integra APIs externas para enriquecer a experiência de uso e aproximar o protótipo de uma aplicação real. As APIs consumidas são:

API / Serviço	Finalidade
Nominatim (OpenStreetMap)	Autocompletar de endereços com filtro para o Brasil (countrycodes=br), retornando latitude e longitude para o cálculo de rota.
OpenRouteService	Cálculo de rota real em GeoJSON (/v2/directions/driving-car) entre as coordenadas do restaurante e do cliente, com distância em km e tempo estimado.

Leaflet (cliente)	Renderização do mapa interativo com marcadores personalizados para restaurante, cliente e motoboy, e visualização da rota com segmentos já percorridos e restantes.
CloudAMQP	Hospedagem do broker RabbitMQ em ambiente de nuvem para acesso pelos serviços em produção.
Prometheus API (/api/v1/query_range)	Consulta de métricas temporais pelo frontend via proxy, alimentando gráficos Chart.js no painel do administrador.

Tabela 1 – APIs externas integradas ao sistema MicroChefs

4.3. Rastreamento Geográfico e Simulação de Entrega

A jornada de acompanhamento de entrega foi enriquecida com a integração de ferramentas geoespaciais e uma simulação de rota ativa baseada em dados geográficos reais:

- Autocompletar de Endereço (Nominatim API): o componente AddressSearchComponent realiza buscas assíncronas com debounce de 350ms, sugerindo endereços completos filtrados por país. As coordenadas obtidas são armazenadas para cálculo de percurso.
- Cálculo de Rota (OpenRouteService): ao iniciar uma entrega, o sistema consome a API para obter a rota real em GeoJSON entre as coordenadas do restaurante e do cliente, extraíndo distância total, tempo estimado e lista de coordenadas. Um fallback calcula rota interpolada em 20 pontos lineares caso a API esteja offline.
- Renderização Visual com Leaflet: o DeliveryMapComponent exibe marcadores personalizados para Restaurante, Cliente e Motoboy, com animação de pulso luminoso via CSS. A rota é desenhada como linha tracejada cinza para o trecho concluído e linha vermelha contínua para o restante.
- Simulação de Deslocamento: na ausência de GPS real, o frontend gerencia um cronômetro (setInterval a cada 60 segundos) que faz a posição do entregador avançar ao longo do vetor de coordenadas da rota, mantendo velocidade média fictícia de 30 km/h até finalizar automaticamente o pedido como ENTREGUE.

4.4. Observabilidade, Escalabilidade e Cache

Para dar suporte à escalabilidade do sistema e garantir monitoramento adequado, a infraestrutura foi expandida com as seguintes soluções:

- Escalabilidade Horizontal com Docker Compose: o script scripts/scale-cozinha.ps1 ajusta o número de réplicas do container cozinha-service. O balanceamento de carga entre as instâncias ocorre automaticamente pelo RabbitMQ via estratégia round-robin na fila cozinha-queue.
- Autodescoberta Dinâmica de Alvos no Prometheus (File Service Discovery): o script refresh-prometheus-targets.ps1 extrai os IPs das réplicas via docker inspect, regrava os arquivos de targets e aciona o reload do Prometheus (POST /-/reload)

sem interrupção de serviço, garantindo que novas réplicas sejam monitoradas imediatamente após o escalonamento.

- **Painel de Controle Nativo com Prometheus API:** o administrador visualiza gráficos interativos com Chart.js direto no frontend Angular, consultando métricas por `query_range` via proxy. As métricas monitoradas incluem: mensagens prontas nas filas (`rabbitmq_queue_messages_ready`), mensagens não confirmadas, consumidores ativos por fila, latência de preparo na cozinha (p50 e p95) e vazão de processamento por minuto.
- **Integração com Grafana:** painéis automáticos são provisionados via arquivos de configuração, exibindo as métricas do Micrometer. Um atalho no painel do administrador direciona para o dashboard do Grafana.
- **Cache Distribuído com Redis:** o `ProdutoService` usa `@Cacheable("produtos")` para acelerar leituras e `@CacheEvict(allEntries = true)` em alterações, exclusões e atualizações de estoque, reduzindo a carga sobre o PostgreSQL em períodos de alta concorrência.

4.5. Configuração de APIs e Infraestrutura

Todas as credenciais e URLs sensíveis são gerenciadas por variáveis de ambiente no Docker Compose e nos arquivos de configuração de cada serviço, nunca versionadas no código-fonte. As configurações de RabbitMQ, as strings de conexão dos bancos de dados e as chaves de API do OpenRouteService são injetadas em tempo de execução. No serviço de usuários em .NET, as coordenadas do restaurante são configuradas globalmente e utilizadas como ponto de origem para todos os cálculos de rota. O proxy do frontend mapeia `/prometheus` para o endereço local do Prometheus, evitando conflitos de CORS.

Variável de Ambiente	Serviço	Finalidade
RABBITMQ_HOST / CLOUDAMQP_URL	Todos os serviços Java e .NET	Endereço do broker RabbitMQ
SPRING_DATASOURCE_URL	Pedido, Produto, Cozinha, DLQ	Conexão com PostgreSQL
ConnectionStrings__DefaultConnection	Usuario Service (.NET)	Conexão com MySQL
REDIS_HOST / spring.redis.host	Produto Service	Endereço do servidor Redis
ORS_API_KEY	Frontend (environment.ts)	Chave da API OpenRouteService
RESTAURANT_LAT / RESTAURANT_LNG	Usuario Service	Coordenadas do restaurante

Tabela 2 – Principais variáveis de ambiente do projeto

5. Resultados e Discussão

O MicroChefs atende ao escopo funcional previsto para uma plataforma acadêmica de restaurante baseada em microserviços. O sistema demonstra, de forma integrada, cadastro de usuários, autenticação com JWT, gerenciamento de perfis, catálogo de produtos, criação de pedidos, validação de estoque, pagamento via Pix, atualização de status por eventos, fila de cozinha, tratamento de mensagens falhas e painéis administrativos com monitoramento ativo.

Do ponto de vista arquitetural, os principais resultados observados são:

- Separação clara entre domínios de negócio com persistência independente por serviço.
- Redução de acoplamento temporal por meio de filas e tópicos no RabbitMQ.
- Controle consistente de transição de status de pedidos, impedindo fluxos inválidos.
- Cache integrado com Redis otimizando operações de leitura no catálogo de produtos.
- Roteamento e permissões baseados em papéis (roles) no frontend em Angular.
- Recuperação e reprocessamento manual de mensagens descartadas em DLQ.
- Simulação dinâmica de entrega georreferenciada consumindo APIs reais (OpenRouteService e OpenStreetMap) com Leaflet.
- Observabilidade híbrida com Prometheus integrado ao frontend (Chart.js) e painéis automáticos no Grafana.
- Suporte a alta carga por meio de escalonamento dinâmico de instâncias com atualização automática de alvos no Prometheus (File Service Discovery).

A integração entre pedido, produto e cozinha é um dos pontos mais relevantes. O pedido somente avança após validação de estoque, e a cozinha somente recebe ordens já confirmadas, reproduzindo um cenário mais próximo de uma operação real.

Em termos de experiência de uso, o frontend organiza a jornada por perfis: o cliente acompanha o pedido com mapa ativo e percurso em tempo real; o motoboy visualiza tarefas e localização; a cozinha recebe a fila de preparo; e o administrador acessa métricas técnicas de mensageria e latência, produtos e usuários. Isso reforça o valor da modelagem por responsabilidade.

Com a consolidação do Prometheus no console de administração e a automação do re-target de microserviços durante o escalonamento, a arquitetura demonstrou robustez operacional em cenários de alta demanda, permitindo monitorar em tempo real a latência de processamento de cada réplica da cozinha e mitigar gargalos operacionais.

6. Conclusões

O projeto MicroChefs foi concluído de forma coerente com a proposta de demonstrar uma aplicação distribuída para restaurante baseada em microserviços. A solução consolidou autenticação, catálogo, pedidos, cozinha, DLQ e frontend em uma arquitetura modular, com

comunicação síncrona e assíncrona, armazenamento segregado e controle de acesso por perfis.

Do ponto de vista funcional, o sistema cobre o fluxo principal de uso de forma completa: o usuário acessa a plataforma, autentica-se, localiza seu endereço via busca textual automatizada com Nominatim, monta o pedido, realiza o pagamento, acompanha a evolução do preparo e monitora a trajetória em tempo real do motoboy no mapa. Em paralelo, os módulos administrativos permitem manutenção de produtos, usuários e monitoramento de performance por Prometheus e Grafana.

6.1. O que não foi implementado

O sistema foi concluído de forma funcional, mas algumas decisões de escopo foram tomadas conscientemente ao longo do desenvolvimento. A principal delas foi o armazenamento em cache das coordenadas de localização do motoboy. Embora o projeto utilize Redis para cachear o catálogo de produtos, a equipe avaliou que persistir as coordenadas do entregador em cache não faria sentido para o contexto do sistema: as posições são efêmeras e mudam a cada ciclo de simulação, de modo que qualquer valor em cache estaria desatualizado quase imediatamente. Por esse motivo, a posição do motoboy é gerenciada inteiramente no estado do frontend, sem persistência ou cacheamento no lado do servidor.

6.2. Dificuldades Encontradas

As principais dificuldades de implementação concentraram-se em:

- Integração confiável entre serviços desenvolvidos em linguagens distintas (Java e C#), exigindo padronização cuidadosa dos contratos de API e dos formatos JSON trocados no RabbitMQ.
- Padronização de estados e transições do pedido para evitar incoerências entre os diferentes consumidores do barramento de eventos.
- Tratamento de falhas de mensagem sem perda de dados, resolvido com a combinação de retry automático (erros de infraestrutura) e encaminhamento imediato à DLQ (erros de dados).
- Visualização de mapas em tempo real, especialmente a sincronização suave do marcador do motoboy ao longo da rota GeoJSON sem causar renderizações excessivas.
- Observabilidade e escalabilidade da aplicação: garantir que o Prometheus reconhecesse automaticamente novas réplicas da cozinha após escalonamento exigiu a criação dos scripts de File Service Discovery e recarga dinâmica de alvos.
- Integração de gráficos Prometheus diretamente no Angular sem conflitos de CORS, solucionada com proxy de desenvolvimento e consultas via `query_range`.

6.3. Trabalhos Futuros

Como trabalhos futuros, recomenda-se:

- Integração com gateways reais de pagamento Pix (ex.: Mercado Pago, PagSeguro, Gerencianet) com recebimento de webhooks assinados para confirmação automática de pagamentos.
- Monitoramento de coordenadas GPS reais do motoboy, por meio de SDK móvel ou app dedicado, substituindo a simulação atual por atualização de posição em tempo real via WebSocket ou SignalR, tornando o rastreamento útil em um aplicativo de uso real.
- Substituição do polling de status do frontend por canais persistentes bidirecionais (WebSockets ou Server-Sent Events), reduzindo latência e consumo de recursos.
- Deploy da infraestrutura em nuvem pública sob orquestração de Kubernetes (K8s), com Horizontal Pod Autoscaler (HPA) baseado em métricas do Prometheus para escalonamento automático da cozinha.
- Implementação de testes automatizados de carga (ex.: k6 ou Gatling) para validar a resiliência do sistema em picos de demanda e calibrar as políticas de retry e DLQ.

Referências

- [1] NEWMAN, Sam. Building Microservices: Designing Fine-Grained Systems. 2. ed. Sebastopol: O'Reilly Media, 2021.
- [2] RABBITMQ. RabbitMQ Documentation. Disponível em: <https://www.rabbitmq.com/documentation.html>. Acesso em: maio 2025.
- [3] PROMETHEUS. Prometheus Documentation. Disponível em: <https://prometheus.io/docs/>. Acesso em: jun. 2025.
- [4] OPENROUTESERVICE. API Documentation. Disponível em: <https://openrouteservice.org/dev/#/api-docs>. Acesso em: jun. 2025.
- [5] DOCKER. Docker Documentation. Disponível em: <https://docs.docker.com/>. Acesso em: maio 2025.
- [6] SPRING. Spring AMQP Reference Documentation. Disponível em: <https://docs.spring.io/spring-amqp/>. Acesso em: maio 2025.